Kevin Küster

Zusammenfassung: Statements

Statements

Anweisungen

Konkrete Berechnung einer Zuweisung (z.B. von einem Ausdruck)
↳ Berechnungsvorschriften; Schrittweise Abarbeitung
↳ Ergebnis in angegebne Speicherstelle schreiben

Befehlssequenzen

Einzelne aufeinanderfolgende Anweisungen

Beispiel

```
a := 1;
b := a + 2;
c := a * b;
d := a * b + c;
```

⇒ Für Wertzuweisung der Labels muss Speicherplatz reserviert werden
⇒ DD (data definition)

```
SEG
JUMP start
a: DD W 0
b: DD W 0
c: DD W 0
d: DD W 0
start:
    MOVE W, I1, a
    ADD  W, a, I2, b
    MULT W a, b, c
    MULT W a, b, d
    ADD  W d, c, d
    HALT
```

Kontrollfluss

Sprünge zur Umsetzung von Wiederholungen und Bedingungen
↳ Turing-Vollständigkeit

Sprünge:

(i) Direkte, unbedingte Sprünge
↳ Genau ein Nachfolgender Befehl

(ii) Direkte, bedingte Sprünge → Wahrheitswert
↳ Nachfolger 1: Bed. erfüllt
Nachfolger 2: Bed. nicht erfüllt (fall through Branch)

Fallunterscheidung

Umsetzungsmöglichkeiten:

(i) Bed. auf "true" prüfen → then branch:

```
    CMP W a, I 0
    JEQ then_branch
    JMP else_branch
then_branch:
    [...]
    JMP posttix_code
else_branch:
    [...]
posttix_code:
    [...]
```

(ii) Bed. auf "false" prüfen → else branch (geeignetere Variante):

```
    CMP W a, I 0
    JNE else_branch
    [...]
    JMP posttix_code
else_branch:
    [...]
posttix_code:
    [...]
```

While-Statement

```
Code Hochsprache:
[...]                          (pretix code)
i := 10;
n := 0;
while i > 0 loop begin         (condition)
   n := n + 1;                 (loop body)
end loop;
[...]                          (posttix code)
```

(i) Bed. der while-Schleife auswerten:
▷ Bed. "true" → loop body
▷ Bed. "false" → posttix code

(ii) Nach Ausführung loop body → Bed. erneut prüfen

Umsetzungsmöglichkeiten:

(i) Prüfung vor Schleifen-Rumpf:

```
    [...]      ;pretix code
while_header:
    CMP W i, I 0
    JNE while_end
    [...]
    JMP while_header
while_end:
    [...]      ; posttix code
```

(ii) "-" nach "-" (Bevorzugt):

```
    [...]      ; pretix code
    JMP while_header
while_header:
    [...]
while_condition:
    CMP W i, I 0
    JEQ while_header
while_end:
    [...]      ; posttix code
```

for - Statement

Code-Hochsprache:

```
[...]                                  (pretix code)
i := 0;
n := 0;
for i in 5 .. 10 loop begin
    n := n + i;                        (loop body)
end loop;
[...]                                  (postfix code)
```

Vorarbeitung:

(i) Initialisierung Zählvariable: $i = [5, 10)$

(ii) Überprüfung: < 10

(iii) Loop body ausführen

(iv) Zählvariable modifizieren, Sprung Schleifenanfang

⇒ Übersetzung analog zu while-Statement:

```
[...]                                  (pretix code)
i := 0;
n := 0;                                (init code)
i := 5;
while i < 10 loop begin
    n := n + i;                        (loop body)
    i := i + 1;                        (loop modifier)
end loop
[...]                                  (postfix code)
```

Beispiel

Code Hochsprache

```
i := 0;
sum := 0;
for i in 1...100 loop begin
    sum := sum + i;
end loop;
```

for → while ⟹

```
i := 0;
sum := 0;
while i < 100 loop begin
    sum := sum + i;
    i := i + 1;
end loop;
```

⟹

```
    i := 0;
    sum := 0;
    goto while_condition;
while_loop:
    sum := sum + i;
    i := i + 1;
while_condition:
    if i < 100 then goto while_loop;
    end if;
```

Assembler ⟹

```
    MOVE W I 0, i
    MOVE W I 0, sum
    JMP while_condition;
while_loop:
    ADD W sum, i, sum
    ADD W i, I 1, i
while_condition:
    if i < 100 then goto while_loop;
    end if;
    CMP W i, I 100
    JLT while_loop
```

Notwendige Datendefinitionen zur Speicherreservierung ergänzen ⟹

```
    SEG
    JMP start
i:  DD W 0
sum: DD W 0
start:
    MOVE W I 0, i
    MOVE W I 0, sum
    JMP while_condition;
[...]
```